

# Process Modeling and Tools

BPMN, Visio, twelve diagram types and SQL for analysts

---

- 01**    **The chain: capability to requirement**  
Capabilities, three process levels, personas, stories, requirements
- 02**    **BPMN and Visio**  
Full notation, modelling rules, tool workflow and validation
- 03**    **Twelve diagram types**  
Purpose, notation, when to use and worked examples for each
- 04**    **SQL essentials for BAs**  
Query mechanics, joins, and five worked Northwind queries

## Week 2 at a glance

*This is the week you stop describing systems in prose and start modelling them. Notation is a language: the point is being understood without a conversation.*

| Course / task                                            | The five things to take from it                                                                                                                  |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Learning BPMN                                            | Events, activities and gateways; sequence versus message flow; pools and lanes; when to use each gateway type; the rules that make a model valid |
| Learning Visio                                           | Templates and stencils; connecting and gluing shapes; cross-functional flowcharts; shape data; BPMN validation; export and share                 |
| Capabilities, processes, personas, stories, requirements | What each artefact is, the three process levels, and how one traces into the next                                                                |
| Twelve diagram types                                     | What question each answers, the notation, and which ones people confuse                                                                          |
| SQL Essential Training (backlog item)                    | SELECT mechanics, filtering, joins, grouping, and reading a schema                                                                               |

### The deliverables

- A BPMN model of the 'reorder past meal' process - Part 2 works one through completely, including the exception path most people forget.
- Five SQL SELECT queries answering business questions against the Northwind sample database - Part 4 works all five, with the reasoning.
- An explanation to your team of capabilities, the three process levels, personas, stories and requirements - Part 1 gives you the structure and a worked example that threads through all five.

#### EXAM / INTERVIEW TIP

When you present to the team, do not walk them through definitions one at a time. Open with a single real example and show it changing shape as it moves down the chain. People remember the thread, not the taxonomy.

# Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Part 1: Capabilities to Requirements</b>                 | <b>4</b>  |
| 1.1 The Chain . . . . .                                     | 4         |
| 1.2 Business Capabilities . . . . .                         | 5         |
| 1.3 Business Processes - the Three Levels . . . . .         | 5         |
| 1.4 Personas . . . . .                                      | 6         |
| 1.5 User Stories and Requirements . . . . .                 | 7         |
| <b>Part 2: BPMN and Visio</b>                               | <b>8</b>  |
| 2.1 Why BPMN . . . . .                                      | 8         |
| 2.2 The Notation . . . . .                                  | 8         |
| 2.3 Rules That Make a Model Valid . . . . .                 | 11        |
| 2.4 Worked Model: Reorder Past Meal . . . . .               | 12        |
| 2.5 Visio Essentials . . . . .                              | 13        |
| <b>Part 3: Twelve Diagram Types</b>                         | <b>16</b> |
| 3.1 The Selector . . . . .                                  | 16        |
| 3.2 BPMN Diagram . . . . .                                  | 17        |
| 3.3 Use Case Diagram . . . . .                              | 17        |
| 3.4 Entity-Relationship Diagram . . . . .                   | 19        |
| 3.5 Data Flow Diagram . . . . .                             | 21        |
| 3.6 Wireframe and Mockup . . . . .                          | 21        |
| 3.7 Swimlane Diagram (Cross-Functional Flowchart) . . . . . | 23        |
| 3.8 Gantt Chart . . . . .                                   | 23        |
| 3.9 Stakeholder Map . . . . .                               | 25        |
| 3.10 State Diagram . . . . .                                | 25        |
| 3.11 Requirements Traceability Matrix . . . . .             | 27        |
| 3.12 Gap Analysis Diagram . . . . .                         | 27        |
| 3.13 Activity Diagram . . . . .                             | 29        |
| 3.14 The Pairs People Confuse . . . . .                     | 29        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Part 4: SQL Essentials for Analysts</b>                | <b>31</b> |
| 4.1 Why This Matters . . . . .                            | 31        |
| 4.2 Anatomy of a SELECT . . . . .                         | 31        |
| 4.3 Filtering, Sorting and NULLs . . . . .                | 32        |
| 4.4 Aggregates and Grouping . . . . .                     | 33        |
| 4.5 Joins . . . . .                                       | 33        |
| 4.6 The Northwind Schema . . . . .                        | 34        |
| 4.7 Five Worked Queries . . . . .                         | 35        |
| 4.8 Mistakes That Produce Wrong Numbers Quietly . . . . . | 38        |
| 4.9 Going Further . . . . .                               | 38        |
| <b>Self-Check Questions</b>                               | <b>39</b> |

## PART 1

# Capabilities to Requirements

*Five artefacts, one chain, and why the links matter more than the definitions*

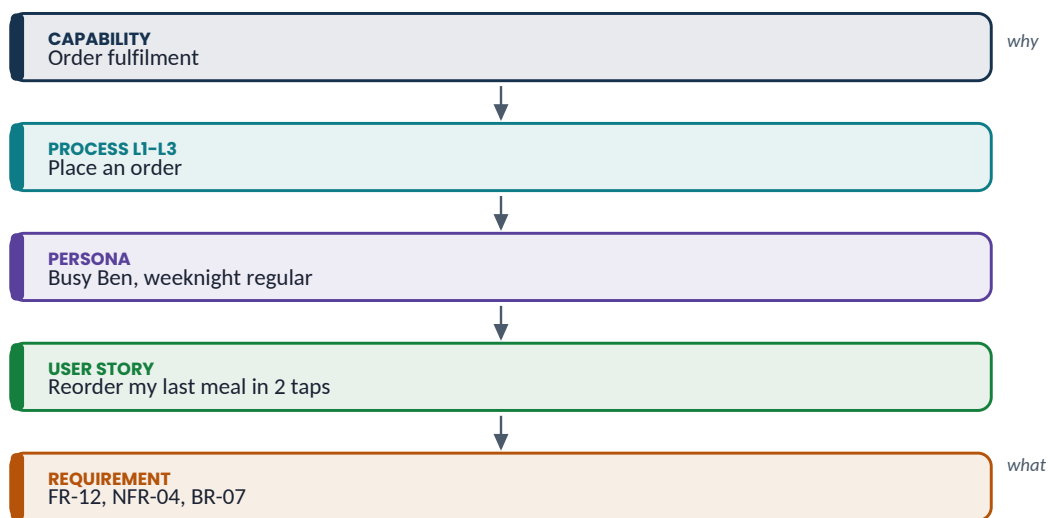
1

## What this part covers

- What a business capability is and why it is not a process
- The three levels of business process and how to decompose
- Personas: what makes one useful rather than decorative
- How user stories sit between process and requirement
- Requirements as the testable end of the chain
- A single example carried through all five artefacts

## 1.1 The Chain

These five artefacts are not alternatives. They are the same need described at decreasing levels of abstraction, and each one answers a question the level above cannot.



*The same need, described five times - abstract at the top, testable at the bottom*

| Artefact         | Answers                                      | Changes how often?      | Owned by                |
|------------------|----------------------------------------------|-------------------------|-------------------------|
| Capability       | What is the business able to do?             | Rarely - years          | Enterprise architecture |
| Business process | How does it do it, step by step?             | Occasionally - quarters | Process owner           |
| Persona          | Who does it, and what do they care about?    | Slowly - annually       | Product / UX            |
| User story       | What does that person need to be able to do? | Constantly - sprints    | Product owner           |

| Artefact    | Answers                                          | Changes how often? | Owned by  |
|-------------|--------------------------------------------------|--------------------|-----------|
| Requirement | What exactly must be true for it to be accepted? | Per item           | BA / team |

**KEY POINT**

The value is in the **links**, not the boxes. If you cannot trace a user story up to a capability, you cannot explain why it is worth building. If you cannot trace a capability down to stories, you cannot explain how it will ever improve.

## 1.2 Business Capabilities

A capability is **what** the business is able to do, stated as a noun phrase and deliberately independent of how it is done, who does it, or what technology supports it. Reorganising a department does not change your capabilities; it changes who performs them.

| This is a capability                    | This is not                   |
|-----------------------------------------|-------------------------------|
| Order fulfilment                        | The order fulfilment team     |
| Payment processing                      | Stripe integration            |
| Customer identity and access management | The login screen              |
| Menu and catalogue management           | Updating menus every Tuesday  |
| Delivery routing and dispatch           | Assign courier via mobile app |

- Capabilities are **stable**. Processes, systems and org structures change around them constantly.
- They are usually arranged in a **capability map** of two or three levels, used to spot duplication, gaps and where investment is concentrated.
- A **heat map** over the capability map (colouring by maturity, cost or pain) is one of the fastest ways to show executives where the problem is.
- Test: if you can put 'the ability to...' in front of it and it still reads naturally, it is probably a capability.

**EXAMPLE**

For a food delivery business, level-1 capabilities might be: Customer engagement, Menu and catalogue management, Order management, Payment processing, Delivery operations, Partner management, Analytics and reporting. **Order management** then decomposes into browse and select, basket management, order placement, order tracking and order amendment - and reorder sits inside order placement.

## 1.3 Business Processes – the Three Levels

Process decomposition gives everyone a shared idea of how zoomed-in a conversation is. The naming varies between organisations, but the three-level structure is near universal.

| Level   | Also called                                 | Shows                                                                                                | Audience                            | Reorder example                                                       |
|---------|---------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------|
| Level 1 | Process landscape, value chain, megaprocess | The handful of end-to-end processes that run the business, on one page                               | Executives                          | Order to delivery                                                     |
| Level 2 | Process map, sub-process                    | The major stages within one L1 process and their sequence and handoffs                               | Process owners, managers            | Place an order (within order to delivery)                             |
| Level 3 | Process flow, activity level, procedure     | The actual step-by-step activities, decisions, roles and systems - this is where BPMN swimlanes live | BA, delivery team, operations staff | Reorder a past meal, including availability and price-change handling |

- Some organisations add **Level 4** - work instructions, screen-by-screen steps, the 'how to do it' a new starter is handed. That is training material, not analysis.
- The rule of thumb: **L1 fits on a page, L2 fits on a page per L1 process, L3 fits on a page per L2 stage**. If your L3 needs three pages, you are at L4.
- Model **as-is** and **to-be** at L3. Comparing them is where the gap analysis, and therefore the requirements, come from.

#### WATCH OUT

The most common decomposition mistake is mixing levels in one diagram - a box reading 'Manage orders' sitting next to a box reading 'Click Confirm'. If two boxes on the same diagram would take wildly different amounts of time, you have mixed levels.

## 1.4 Personas

A persona is an evidence-based archetype of a user group: a compact summary of who they are, what they are trying to achieve, and what gets in their way. Its job is to stop the team designing for themselves.

| Element        | What good looks like                                              |
|----------------|-------------------------------------------------------------------|
| Name and photo | Memorable and human - 'Busy Ben', not 'User Type B'               |
| Context        | Where, when and on what device they do this                       |
| Goals          | What they are trying to achieve, in their words                   |
| Frustrations   | What currently gets in the way, with a real quote if you have one |
| Behaviour      | Frequency, expertise level, tolerance for complexity              |
| Constraints    | Accessibility needs, connectivity, time pressure, language        |
| Anti-goals     | What they do not care about - as useful as what they do           |

**EXAMPLE**

**Busy Ben, 34, weeknight regular.** Orders two to three times a week, almost always the same two restaurants. Phone, one-handed, between 5.30 and 7pm, usually doing something else. **Goal:** food ordered in under a minute. **Frustration:** 'I know exactly what I want and it still takes three minutes.' **Anti-goal:** he does not want to discover new restaurants - browse features are noise to him.

- Three to five personas is usually enough. More and nobody remembers them.
- A persona built from opinion rather than research is worse than none - it launders guesses as evidence.
- Include at least one persona with an **accessibility** or **low connectivity** constraint, or those requirements will never surface.
- Personas feed directly into the 'as a...' clause of your stories. If your stories all start 'as a user', you have no personas.

## 1.5 User Stories and Requirements

A user story sits between the process and the requirement. It names a person, a capability they need, and the benefit. The requirement is the testable statement that results from the conversation the story triggers.

|          | User story                                           | Requirement                                                    |
|----------|------------------------------------------------------|----------------------------------------------------------------|
| Purpose  | Trigger a conversation about a need                  | State precisely what must be true                              |
| Form     | As a [persona] I want [capability] so that [benefit] | [Condition] the [actor] shall [action] [object]<br>[qualifier] |
| Detail   | Deliberately incomplete                              | Complete enough to build and test                              |
| Lifespan | Discarded once delivered                             | Often retained as documentation of the product                 |
| Owned by | Product owner                                        | BA, agreed with the team                                       |

**KEY POINT**

A story is **not** a small requirement. It is a promise to have a conversation. The requirement is what the conversation produces - which is why acceptance criteria, not the story card, are what the developer builds against.

### Explaining the chain to your team – a script that works

1. Start at the bottom with something concrete: read out one acceptance criterion.
2. Ask: why does that criterion exist? Show the story it belongs to.
3. Ask: who is that persona and why do they care? Show the persona card.
4. Ask: where does this sit in how we actually work? Show the L3 process model.
5. Zoom out twice - L2, then L1 - and land on the capability.
6. Then reverse it in thirty seconds: capability, process, persona, story, requirement. People remember the round trip.



## PART 2

# BPMN and Visio

*The notation that makes a process model unambiguous, and the tool to draw it in*

# 2

## What this part covers

- Why BPMN exists and its three levels of use
- Events, activities and gateways in full
- Sequence flow, message flow, pools and lanes
- Data objects, stores, annotations and artifacts
- The rules that make a model valid, and the mistakes that break it
- A complete worked model of the reorder process
- Visio: templates, stencils, gluing, containers, shape data and validation

## 2.1 Why BPMN

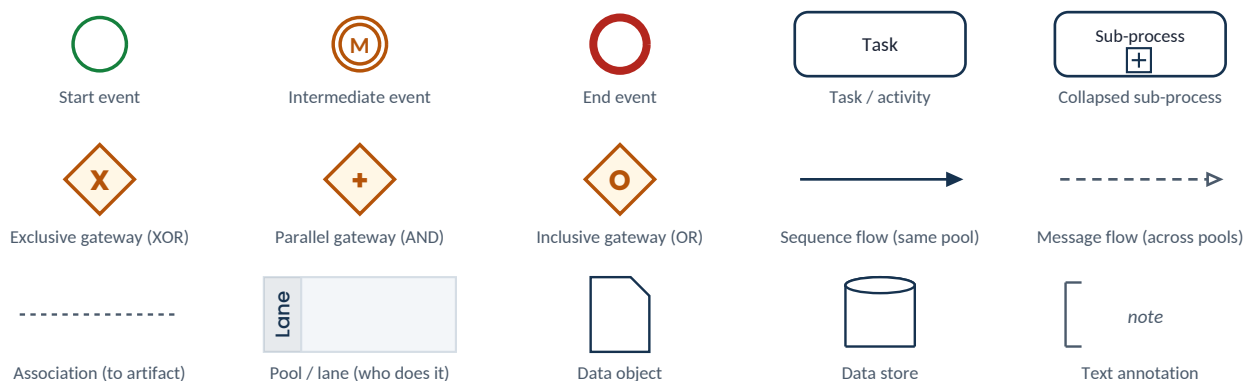
A basic flowchart is fine until two people read it differently. BPMN (Business Process Model and Notation, an OMG standard) fixes the meaning of every shape, so a model means the same thing to a business analyst, a process owner and a developer.

| Level of use | Elements used                                                                                                        | Audience                                                        |
|--------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Descriptive  | Start and end events, task, sub-process, exclusive and parallel gateways, sequence flow, pools and lanes, annotation | Business stakeholders - covers around 80 percent of real models |
| Analytic     | Adds message, timer and error events, event-based and inclusive gateways, data objects and stores, loops             | BAs and process analysts modelling exceptions properly          |
| Executable   | Full specification including technical attributes and expressions                                                    | Process automation engineers                                    |

### EXAM / INTERVIEW TIP

Aim for descriptive level plus message and timer events. That is enough to model almost any business process precisely, and it stays readable by the people who have to confirm it is correct - which is the whole point.

## 2.2 The Notation



*The elements you will use in nearly every model*

## Events – something happens

| Event        | Border            | Meaning                              | Example                         |
|--------------|-------------------|--------------------------------------|---------------------------------|
| Start        | Thin single line  | What triggers the process            | Customer opens order history    |
| Intermediate | Double line       | Something happens during the process | Payment confirmation received   |
| End          | Thick single line | A result of the process              | Order placed                    |
| Message      | Envelope symbol   | A message is sent or received        | Restaurant accepts the order    |
| Timer        | Clock symbol      | Time-based trigger or delay          | No response within 5 minutes    |
| Error        | Lightning symbol  | Something went wrong                 | Payment declined                |
| Signal       | Triangle symbol   | A broadcast that anyone can catch    | Restaurant closes for the night |

- A **boundary event** is drawn on the edge of an activity and interrupts it - the standard way to model timeouts and errors without cluttering the main flow.
- Every process should have **at least one start and one end event**. Multiple end events are fine and often clearer than merging everything back together.

## Activities – something is done

| Activity type         | Meaning                                                                |
|-----------------------|------------------------------------------------------------------------|
| Task                  | A single unit of work that is not broken down further at this level    |
| User task             | A person does it with system support - shown with a person marker      |
| Service task          | A system does it with no human involvement - gear marker               |
| Manual task           | A person does it with no system involved at all - hand marker          |
| Business rule task    | A decision is evaluated against a rule set - table marker              |
| Sub-process           | A compound activity, collapsed with a plus marker or expanded in place |
| Call activity         | Invokes a process defined elsewhere - reuse rather than duplication    |
| Loop / multi-instance | Repeats; multi-instance can run sequentially or in parallel            |

**WATCH OUT**

Name activities **verb plus object**: 'Validate item availability', not 'Validation' or 'Availability'. A noun-only box hides whether it is a step, a state or a document, and reviewers will read it three different ways.

## Gateways – the path splits or joins

| Gateway         | Symbol             | Behaviour                                        | Use when                                                                     |
|-----------------|--------------------|--------------------------------------------------|------------------------------------------------------------------------------|
| Exclusive (XOR) | X                  | Exactly one outgoing path is taken               | Mutually exclusive conditions - available or not available                   |
| Parallel (AND)  | Plus               | All outgoing paths are taken simultaneously      | Independent work that can happen at once - charge card and notify restaurant |
| Inclusive (OR)  | Circle             | One or more paths are taken, based on conditions | Conditions that can overlap - item unavailable and price changed             |
| Event-based     | Pentagon in circle | The path is decided by which event happens first | Waiting for whichever comes first - restaurant accepts or timeout expires    |
| Complex         | Asterisk           | Behaviour described in an annotation             | Rare; usually a sign the model needs restructuring                           |

- A gateway does not **do** work. It routes. If you find yourself wanting to write a verb in a diamond, you need a task before it.
- Label the gateway with a **question** and each outgoing path with the **answer**: 'All items available?' with Yes and No.
- Every exclusive split should have a matching join, or a clear reason why the paths end separately.
- **Never leave an unlabelled default path.** If a condition is not met and no path applies, the process silently stops - the single most common modelling bug.

## Flows, pools and lanes

| Element                                | Rule                                                                               |
|----------------------------------------|------------------------------------------------------------------------------------|
| Sequence flow (solid arrow)            | Connects activities within a single pool. Never crosses a pool boundary            |
| Message flow (dashed arrow, open head) | Connects two pools. Represents communication between separate participants         |
| Association (dotted line)              | Links an artifact such as an annotation or data object to a flow element           |
| Pool                                   | One participant or organisation - the customer, your company, the payment provider |
| Lane                                   | A role, department or system within a pool - who performs the activity             |

**KEY POINT**

The rule people fail exams on: **sequence flow stays inside a pool; message flow goes between pools.** If you have drawn a solid arrow from your company's pool into the customer's pool, the model is invalid. Use a message flow.

## Artifacts and data

- **Data object** - a document or record that a task produces or consumes, such as the order confirmation.
- **Data store** - persistent data the process reads or writes, such as the order database.
- **Text annotation** - a note for the reader; carries no execution meaning but is often the most useful thing on the page.
- **Group** - a dashed box drawn around related elements for emphasis; does not affect the flow.

## 2.3 Rules That Make a Model Valid

1. Every process has at least one start event and at least one end event.
2. Every flow object is connected - no orphans floating on the canvas.
3. Sequence flow never crosses a pool boundary; message flow always does.
4. Gateways route, they do not perform work.
5. Every gateway path is labelled, and a default path exists where conditions could all fail.
6. Activities are named verb plus object.
7. One diagram holds one level of detail.
8. Flow reads left to right, top to bottom, wherever possible.
9. Nothing dead-ends: every path reaches an end event.

## The mistakes reviewers look for

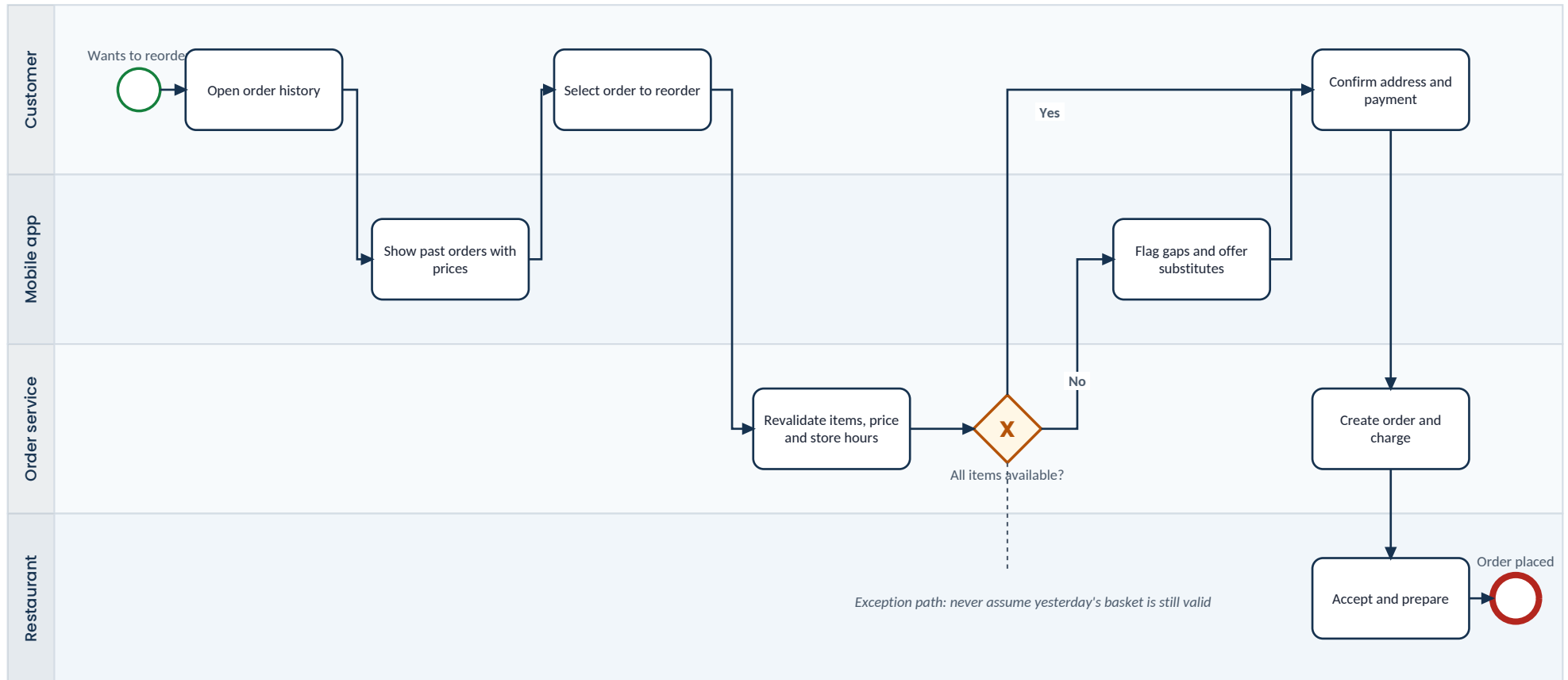
| Mistake                         | Why it matters                                                                                      |
|---------------------------------|-----------------------------------------------------------------------------------------------------|
| Happy path only                 | The exceptions are where the requirements are. A model with no exception path has not been analysed |
| Sequence flow between pools     | Invalid BPMN; hides that a real handoff and a real delay exist                                      |
| Unlabelled gateway paths        | Nobody can tell which condition leads where                                                         |
| Mixed levels of detail          | 'Manage orders' next to 'Click Confirm' makes the model unreadable                                  |
| Lanes named after people        | Name the role, not Sarah. Sarah will change jobs                                                    |
| Diagram spanning three pages    | Collapse a section into a sub-process and model it separately                                       |
| No annotation on the tricky bit | The one thing a reader will misinterpret is exactly the thing worth annotating                      |

### EXAM / INTERVIEW TIP

Model the happy path first and get it agreed. Then go back through and ask, at every single step, 'what could go wrong here?' The second pass is where you earn your fee - and where the requirements nobody thought of come from.

## 2.4 Worked Model: Reorder Past Meal

Four participants as lanes, one exclusive gateway, one exception path. This is the descriptive level of BPMN and it is enough to specify the feature.



Level 3 process model - 'Reorder a past meal', to-be

## Reading the model

| Element                                                | Why it is there                                                                                                          |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Four lanes                                             | Customer, mobile app, order service and restaurant are genuinely different actors. Separating them exposes every handoff |
| 'Revalidate items, price and store hours'              | The single most important task in the model. Without it, the process silently assumes yesterday's basket is still valid  |
| Exclusive gateway 'All items available?'               | Exactly one path is taken; the two paths are labelled Yes and No                                                         |
| 'Flag gaps and offer substitutes' in the app lane      | The exception path. Most first drafts of this model omit it entirely                                                     |
| Both paths converging on 'Confirm address and payment' | The customer always confirms - the exception path adds a step, it does not skip checkout                                 |
| End event in the restaurant lane                       | The process ends when the restaurant accepts, not when the customer taps pay                                             |

### KEY POINT

Every requirement in your week 1 BRD should be visible somewhere in this model. BR-02 (inform of changes before payment) is the gateway and the substitution task. If a business requirement has no counterpart in the process model, one of the two artefacts is wrong.

## Questions the model makes people answer

- What counts as 'available'? Item in stock, or restaurant open, or both?
- If the price changed, does the customer see it before or after the substitution screen?
- Who decides the substitute - the app, the restaurant, or the customer?
- What happens if the restaurant declines after the customer has paid?
- Is the basket held if the customer abandons at the substitution screen?

### EXAMPLE

That last list is the real output of the modelling exercise. You drew the diagram to expose five questions nobody had asked. Answering them produces requirements; the diagram is just how you found them.

## 2.5 Visio Essentials

### Getting set up correctly

1. File, New, then search templates for **BPMN Diagram** - not Basic Flowchart. It loads the correct stencil and enables validation.
2. For a swimlane view without full BPMN, use **Cross-Functional Flowchart**.
3. The **Shapes** pane on the left holds stencils; pin the BPMN Basic Shapes stencil.
4. Set the page orientation to landscape before you start - process models grow sideways.

## The five techniques that save the most time

| Technique    | How                                                                              | Why                                                                                                           |
|--------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| AutoConnect  | Hover a shape, blue arrows appear, click to place and connect the next shape     | Places and connects in one action; keeps spacing consistent                                                   |
| Dynamic glue | Drag a connector to the middle of a shape until the whole shape highlights green | The connector re-routes when you move shapes. Point glue (a green square on one handle) does not              |
| Containers   | Insert, Container, or use the swimlane shapes; drag shapes inside                | Members move with the container; add a lane and everything reflows                                            |
| Shape Data   | Right-click a shape, Data, Define Shape Data                                     | Attach owner, system, cycle time or requirement ID to each step - turns a picture into structured information |
| Validation   | Process tab, Check Diagram                                                       | Flags BPMN rule breaches such as a sequence flow crossing a pool                                              |

### WATCH OUT

**Dynamic versus point glue** is the difference between a diagram you can rearrange and one you have to redraw. If your connectors detach or stay stuck to the wrong spot when you move a box, you used point glue. Delete the connector and redraw it onto the body of the shape, not onto a handle.

## Worth knowing

- **Themes and variants** (Design tab) restyle the whole diagram consistently - use one, do not colour shapes individually.
- **Layers** let you show or hide categories, such as as-is versus to-be steps on one page.
- **Link data to shapes** (Data tab) imports an Excel table and binds rows to shapes, with data graphics displayed on each.
- **Hyperlinks** on a sub-process shape can jump to the page holding its detail - this is how you keep an L2 map linked to its L3 flows.
- Export to PDF for review; export to PNG for slides. Send the .vsdx only to people who have Visio.
- **Free alternatives** if you do not have a licence: draw.io (diagrams.net), Lucidchart, Bizagi Modeler and Camunda Modeler. Bizagi and Camunda enforce BPMN rules more strictly than Visio does, which is useful while you are learning.

## Shortcuts worth memorising

| Shortcut     | Action           | Shortcut     | Action              |
|--------------|------------------|--------------|---------------------|
| Ctrl+Shift+P | Pointer tool     | Ctrl+3       | Connector tool      |
| Ctrl+1       | Pointer / select | Ctrl+2       | Text tool           |
| F2           | Edit shape text  | Ctrl+Shift+Y | Duplicate selection |
| Ctrl+Shift+F | Bring to front   | Ctrl+Shift+B | Send to back        |

| Shortcut     | Action          | Shortcut        | Action              |
|--------------|-----------------|-----------------|---------------------|
| Ctrl+Shift+H | Flip horizontal | Ctrl+L / Ctrl+R | Rotate left / right |



## PART 3

# Twelve Diagram Types

*What each one answers, how to read it, and which pairs people confuse*

3

## What this part covers

- A one-page selector: which diagram answers which question
- All twelve types with notation, use cases and worked examples
- The pairs that get confused: BPMN and activity, DFD and BPMN, use case and story
- How the diagrams connect to each other on a real feature

## 3.1 The Selector

*Pick the diagram by the question you need answered, not by the one you know how to draw.*

| If you need to show...                                    | Use                                   | Level      |
|-----------------------------------------------------------|---------------------------------------|------------|
| How work flows across roles, with decisions and handoffs  | BPMN diagram                          | Process    |
| The same, but simpler and without BPMN rules              | Swimlane / cross-functional flowchart | Process    |
| Who uses the system and for what goals                    | Use case diagram                      | Scope      |
| The detailed steps of one interaction, including branches | Activity diagram                      | Behaviour  |
| What data exists and how it relates                       | Entity-relationship diagram           | Data       |
| Where data comes from, goes to, and is stored             | Data flow diagram                     | Data       |
| The states a thing moves through and what is legal        | State diagram                         | Behaviour  |
| What a screen contains and how it is laid out             | Wireframe / mockup                    | Interface  |
| Who is affected and how to engage them                    | Stakeholder map                       | People     |
| When work happens and what depends on what                | Gantt chart                           | Schedule   |
| The distance between today and the target                 | Gap analysis diagram                  | Strategy   |
| That every requirement is built and tested                | Requirements traceability matrix      | Governance |

### KEY POINT

For your feature you do not need all twelve. A realistic set for 'reorder past meal' is: **context DFD** to fix scope, **BPMN** for the process, **use case** for who does what, **state diagram** for the order lifecycle, **wireframes** for the screens, **ERD** for the data, and an **RTM** to hold it together. That is seven, and each one earns its place.

## 3.2 BPMN Diagram

|                          |                                                                                                                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | How does work flow across roles and systems, including exceptions?                                                                                                                  |
| <b>Key notation</b>      | Circles for events, rounded rectangles for tasks, diamonds for gateways, pools and lanes for participants, solid sequence flow within a pool and dashed message flow between pools. |
| <b>Reach for it when</b> | You need an unambiguous, standards-based model of a process that developers, process owners and auditors will all read.                                                             |

### EXAM / INTERVIEW TIP

BPMN's advantage over a plain flowchart is that every symbol has a fixed meaning. Its cost is that you must respect the rules - the moment you draw a solid arrow between two pools, you have a picture, not a model.

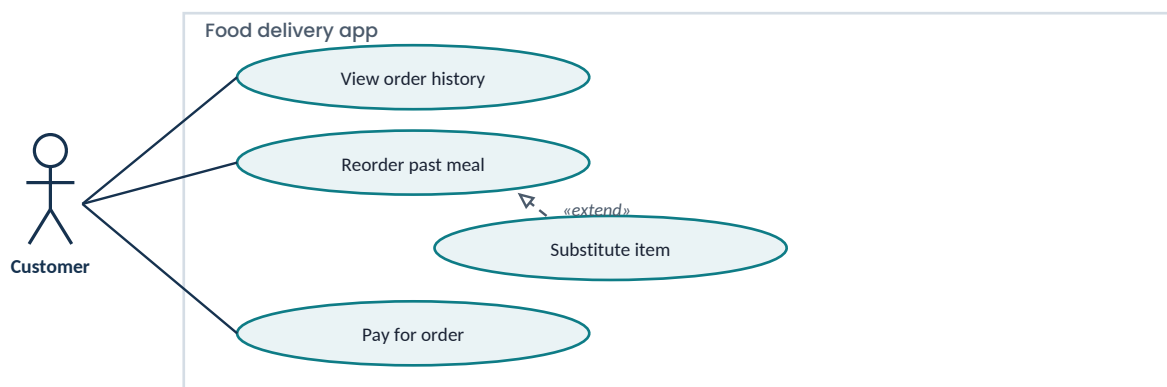
Full treatment, notation reference and a complete worked model are in Part 2.

## 3.3 Use Case Diagram

|                          |                                                                                                                                                                                      |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | Who interacts with the system, and what goals do they pursue?                                                                                                                        |
| <b>Key notation</b>      | Stick figures for actors, ellipses for use cases, a rectangle for the system boundary, solid lines for associations, and dashed arrows labelled include or extend for relationships. |
| <b>Reach for it when</b> | You need to agree scope quickly - what is inside the system and who it serves - before arguing about detail.                                                                         |

### EXAM / INTERVIEW TIP

The diagram deliberately shows almost no detail. Its power is the **boundary box**: everything inside is your responsibility, everything outside is not. Draw it in the first week of any project.



Use case diagram - reorder feature, with 'Substitute item' extending 'Reorder past meal'

| Relationship | Meaning                                            | Example                                  |
|--------------|----------------------------------------------------|------------------------------------------|
| Association  | This actor participates in this use case           | Customer - Reorder past meal             |
| Include      | The base use case always performs the included one | Reorder past meal includes Pay for order |

| Relationship   | Meaning                                                            | Example                                                                     |
|----------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Extend         | The extending use case sometimes adds behaviour, under a condition | Substitute item extends Reorder past meal, only when an item is unavailable |
| Generalisation | One actor or use case is a specialised form of another             | Registered customer is a specialisation of Customer                         |

**WATCH OUT**

Include and extend are the most-failed exam topic. **Include is always; extend is sometimes.** The arrow for both points *towards the base use case* for include, and *from the extension to the base* for extend - draw them the wrong way round and the meaning reverses.

## 3.4 Entity–Relationship Diagram

|                          |                                                                                                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | What things does the business care about, and how do they relate?                                                                                                 |
| <b>Key notation</b>      | Rectangles for entities, lines for relationships, crow's foot notation for cardinality, attributes listed inside the entity with primary and foreign keys marked. |
| <b>Reach for it when</b> | You need to define data structure, spot missing entities, or understand a database before writing SQL against it.                                                 |

### EXAM / INTERVIEW TIP

Read cardinality out loud in both directions: 'one customer places many orders; one order is placed by exactly one customer.' If a sentence sounds wrong when spoken, the model is wrong.

### Crow's foot cardinality

| Notation               | Reads as                            |
|------------------------|-------------------------------------|
| Single bar             | Exactly one (mandatory)             |
| Circle                 | Zero (optional)                     |
| Crow's foot            | Many                                |
| Circle and crow's foot | Zero or many - optional, unlimited  |
| Bar and crow's foot    | One or many - mandatory, unlimited  |
| Circle and bar         | Zero or one - optional, at most one |

### Worked example – the reorder data model

| Entity     | Key attributes                                                                | Relationships                                                          |
|------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Customer   | customer_id (PK), name, email, phone                                          | Places zero or many Orders                                             |
| Restaurant | restaurant_id (PK), name, opening_hours                                       | Has one or many Menu Items                                             |
| Menu Item  | item_id (PK), restaurant_id (FK), name, price, is_available                   | Belongs to exactly one Restaurant; appears in zero or many Order Lines |
| Order      | order_id (PK), customer_id (FK), restaurant_id (FK), placed_at, status, total | Placed by exactly one Customer; contains one or many Order Lines       |
| Order Line | order_line_id (PK), order_id (FK), item_id (FK), quantity, unit_price         | Belongs to exactly one Order and references exactly one Menu Item      |

**EXAMPLE**

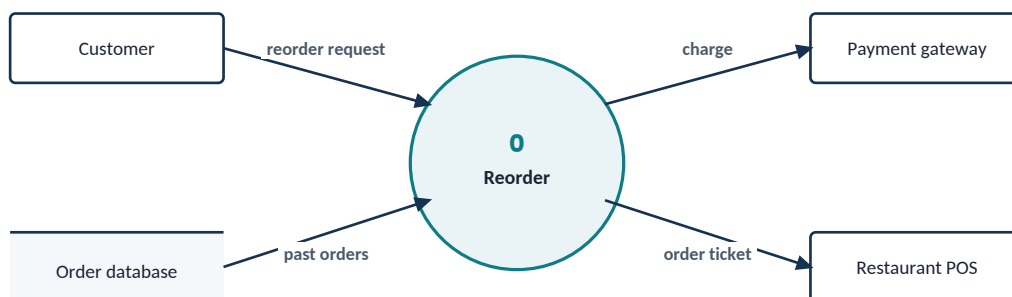
The reorder feature lives or dies on one modelling decision visible above: **Order Line stores unit\_price** as well as referencing the menu item. That is how you can tell the customer the price has changed - you have what they paid last time *and* what it costs now. Miss that attribute and US-04 from week 1 is impossible to build.

## 3.5 Data Flow Diagram

|                          |                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | Where does data come from, what transforms it, and where is it stored?                                                                                                                       |
| <b>Key notation</b>      | Circles or rounded rectangles for processes, open-ended rectangles for data stores, squares for external entities, labelled arrows for data flows. No decisions, no timing, no control flow. |
| <b>Reach for it when</b> | You need to agree system scope, or trace what data crosses a boundary - especially useful for privacy, integration and migration analysis.                                                   |

### EXAM / INTERVIEW TIP

A DFD shows **data movement**, never sequence. If you catch yourself wanting a diamond or a 'then', you want a BPMN or activity diagram instead.



Level 0 context diagram - the reorder process and everything it exchanges data with

| Level             | Shows                                                                     | Purpose                                      |
|-------------------|---------------------------------------------------------------------------|----------------------------------------------|
| Context (level 0) | The whole system as one process with external entities                    | Agree the scope boundary in a single picture |
| Level 1           | The major processes inside, numbered 1, 2, 3, and the stores between them | Decompose the system into its main functions |
| Level 2           | One level-1 process broken down further, numbered 1.1, 1.2                | Detail where it is needed, and only there    |

- **Balancing** is the key rule: the data flows in and out of a process at one level must match the flows in and out of its decomposition at the next.
- Two notations exist - **Yourdon-DeMarco** (circles) and **Gane-Sarson** (rounded rectangles). Pick one and stay consistent.
- Common error: a data store that only ever has arrows going in, or only ever out. Data that is written and never read, or read and never written, is a bug in the model or in the system.

## 3.6 Wireframe and Mockup

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| <b>Answers</b> | What is on this screen, where, and what can the user do with it? |
|----------------|------------------------------------------------------------------|

|                          |                                                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Key notation</b>      | Grey boxes, placeholder text, no branding or colour. A mockup adds visual design; a prototype adds interaction.    |
| <b>Reach for it when</b> | Stakeholders cannot describe what they want in the abstract, or you need to confirm field-level requirements fast. |

**EXAM / INTERVIEW TIP**

Deliberately keep wireframes ugly. The moment they look finished, feedback shifts from 'that field is missing' to 'I don't like the blue' - and you lose the conversation you actually needed.

| Fidelity  | What it is                                           | Use it to                                          |
|-----------|------------------------------------------------------|----------------------------------------------------|
| Sketch    | Pen on paper or whiteboard                           | Explore several ideas in fifteen minutes           |
| Wireframe | Greyscale layout, real content structure, no styling | Agree what is on the screen and its hierarchy      |
| Mockup    | Styled, on-brand, static                             | Agree the visual design                            |
| Prototype | Clickable, simulates the flow                        | Test whether people can actually complete the task |

**What a BA annotates on a wireframe**

- Field-level rules: mandatory or optional, format, maximum length, valid values.
- Validation messages and where they appear.
- Empty states, loading states and error states - the three that are always forgotten.
- What each action does and where it goes next.
- Which elements are visible to which role.

**EXAMPLE**

On the reorder screen the annotations that matter are: what shows when history is empty; how an unavailable item is displayed; how a price change is displayed; what happens if the customer already has a basket. Those four annotations are the acceptance criteria from week 1 drawn rather than written.

## 3.7 Swimlane Diagram (Cross-Functional Flowchart)

|                          |                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | Who does each step, and where does work hand off between them?                                                                                                        |
| <b>Key notation</b>      | Horizontal or vertical lanes labelled by role or department; standard flowchart shapes inside - rectangle for a step, diamond for a decision, oval for start and end. |
| <b>Reach for it when</b> | You need to expose handoffs, delays and unclear ownership without imposing the full BPMN rule set on a business audience.                                             |

### EXAM / INTERVIEW TIP

Count the arrows that cross a lane boundary. Each crossing is a handoff, and handoffs are where delay, error and blame accumulate. A process with fourteen crossings has a design problem, not a staffing problem.

|          | Swimlane flowchart                      | BPMN with lanes                                   |
|----------|-----------------------------------------|---------------------------------------------------|
| Rule set | Loose - a convention                    | Formal standard with defined semantics            |
| Symbols  | Rectangle, diamond, oval                | Events, activities, gateways, artifacts           |
| Audience | Anyone                                  | Anyone, but precision is preserved for developers |
| Best for | A quick shared picture of who does what | A model that will drive build or automation       |

### KEY POINT

Practically: a swimlane flowchart is what you draw on the whiteboard in the workshop; BPMN is what you draw afterwards when it has to be right. Both are legitimate - just do not call a loose flowchart a BPMN model.

## 3.8 Gantt Chart

|                          |                                                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | When does each piece of work happen, and what depends on what?                                                                                      |
| <b>Key notation</b>      | Horizontal bars on a time axis; arrows for dependencies; diamonds for milestones; sometimes a shaded critical path and percentage-complete shading. |
| <b>Reach for it when</b> | You need to communicate a schedule, show dependencies, or explain why a delay in one place moves a date somewhere else.                             |

### EXAM / INTERVIEW TIP

The BA usually does not own the Gantt - the PM does - but you will be asked which analysis activities go on it. Put elicitation workshops, review cycles and sign-off gates on it, because those consume stakeholder time and stakeholders need warning.

| Term       | Meaning                                                            |
|------------|--------------------------------------------------------------------|
| Dependency | Finish-to-start is the common one: B cannot start until A finishes |



| Term          | Meaning                                                                |
|---------------|------------------------------------------------------------------------|
| Milestone     | A zero-duration marker for a significant point, such as sign-off       |
| Critical path | The longest chain of dependent tasks; any slip here slips the end date |
| Float / slack | How long a task can slip without moving the end date                   |
| Baseline      | The agreed original plan, kept for comparison against actuals          |

**WATCH OUT**

In an agile context a detailed Gantt of user stories is an anti-pattern - it re-fixes scope that the whole approach exists to keep flexible. Use a roadmap of outcomes by quarter instead, and keep Gantt charts for genuinely sequential work such as migrations, hardware and regulatory deadlines.

### 3.9 Stakeholder Map

|                          |                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | Who is affected, how much power and interest do they have, and how should we engage them?                                                                                       |
| <b>Key notation</b>      | Most often a two-by-two grid of power against interest. Alternatives: an onion diagram showing distance from the solution, or an influence network showing who listens to whom. |
| <b>Reach for it when</b> | At the start of any initiative, and again whenever the organisation reorganises or a key person changes.                                                                        |

#### EXAM / INTERVIEW TIP

The genuinely useful version records **attitude** as well as position - supportive, neutral or resistant. A high-power, high-interest, resistant stakeholder is the single biggest risk on most projects, and no power-interest grid on its own will show you that.

|            | Low interest   | High interest  |
|------------|----------------|----------------|
| High power | Keep satisfied | Manage closely |
| Low power  | Monitor        | Keep informed  |

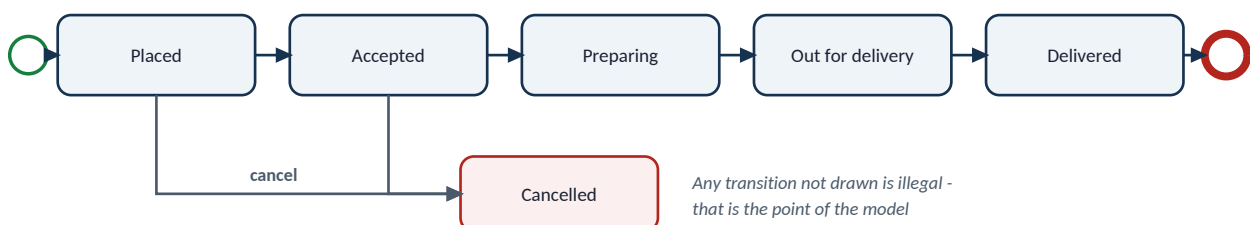
Week 3 covers stakeholder analysis and engagement in depth, including the salience model and communication planning.

### 3.10 State Diagram

|                          |                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | What states can this thing be in, and which transitions are legal?                                                                                                       |
| <b>Key notation</b>      | Rounded rectangles for states, arrows labelled with the triggering event for transitions, a filled circle for the initial state and a ringed circle for the final state. |
| <b>Reach for it when</b> | Anything with a lifecycle and a status field: order, claim, ticket, application, account, subscription.                                                                  |

#### EXAM / INTERVIEW TIP

The value is in the transitions you **do not** draw. If there is no arrow from Delivered back to Preparing, you have just specified that the system must prevent it - which is a requirement nobody would have written in prose.



State diagram - order lifecycle, with cancellation only legal before preparation begins

- Name states as **adjectives or past participles** (Placed, Accepted, Delivered), not verbs. A verb is an activity and belongs on a process model.

- Label every transition with its trigger: cancel, accept, dispatch, timeout.
- Ask for each pair of states: can it go from this one to that one? The answers are requirements.
- Guard conditions can be added in square brackets on a transition, for example cancel when the restaurant has not yet accepted.

## 3.11 Requirements Traceability Matrix

|                          |                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | Is every requirement built and tested, and is everything built traceable to a requirement?                                                    |
| <b>Key notation</b>      | A table, not a picture. Columns for requirement ID, description, source, priority, linked design, linked test cases and status.               |
| <b>Reach for it when</b> | Any project with governance, audit or regulatory obligations - and any project where you will need to answer 'what breaks if we change this?' |

### EXAM / INTERVIEW TIP

The RTM is the only artefact that makes **impact analysis** possible. Without it, every change request is answered with a guess. Build it as you go; reconstructing one at the end takes days and nobody thanks you.

| ID    | Requirement                                             | Source                  | Story              | Test           | Status         |
|-------|---------------------------------------------------------|-------------------------|--------------------|----------------|----------------|
| BR-01 | Re-create a previous order without re-selecting items   | Customer interviews     | US-01, US-02       | TC-01 to TC-06 | Approved       |
| BR-02 | Inform of availability and price changes before payment | Support ticket analysis | US-03, US-04       | TC-07 to TC-14 | In development |
| BR-03 | Amend the re-created basket before paying               | Usability test          | US-05              | TC-15 to TC-18 | Approved       |
| BR-05 | Do not bypass age, allergen or payment controls         | Compliance              | Definition of Done | TC-40, TC-41   | Approved       |

### KEY POINT

**Forward** traceability reads left to right: every requirement reaches a test. **Backward** traceability reads right to left: every test and every built item traces to an agreed requirement. The second direction is how you catch work nobody asked for.

## 3.12 Gap Analysis Diagram

|                          |                                                                                                                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | What is the distance between where we are and where we want to be, and what must change to close it?                                                                                                 |
| <b>Key notation</b>      | Usually two columns - current state and future state - with the gap and the required actions between them. Sometimes drawn as a bridge, a staircase or a spider chart of capability maturity scores. |
| <b>Reach for it when</b> | At the start of an initiative, to derive scope from evidence rather than opinion.                                                                                                                    |

### EXAM / INTERVIEW TIP

Every item in your scope should trace to a gap. Anything in scope that closes no gap is a candidate for deletion, and that argument is much easier to win with a diagram than with an opinion.

| Dimension  | Current state                                         | Future state                                                 | Gap and action                                      |
|------------|-------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------|
| Process    | Repeat customers rebuild the basket item by item      | One action re-creates the basket from history                | New reorder step; revalidation step added           |
| Data       | Order history retained but read-only                  | Order history readable and re-usable by the ordering service | Expose history API; store unit price on order lines |
| Technology | No item-level availability check at basket build      | Availability checked before the basket is shown              | Integrate menu availability service                 |
| People     | Support handles 'price changed' complaints reactively | Customers see changes before paying                          | Update support scripts; train before launch         |
| Measures   | Repeat rate not attributed to a journey               | Reorder journeys tracked end to end                          | Add analytics events on the reorder path            |

### 3.13 Activity Diagram

|                          |                                                                                                                                                                                                                |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Answers</b>           | What are the detailed steps of this behaviour, including branches, loops and parallel work?                                                                                                                    |
| <b>Key notation</b>      | UML notation: rounded rectangles for actions, a filled circle for start, a ringed circle for end, diamonds for decision and merge, thick bars for fork and join, optional partitions that look like swimlanes. |
| <b>Reach for it when</b> | You need to specify the internal logic of one use case or algorithm in detail, particularly where there is concurrency.                                                                                        |

#### EXAM / INTERVIEW TIP

Fork and join is what an activity diagram does better than most notations: the thick bar means all branches run in parallel and all must complete before the flow continues. That is hard to say clearly in prose and easy to draw.

|                     | Activity diagram                           | BPMN diagram                                          |
|---------------------|--------------------------------------------|-------------------------------------------------------|
| Origin              | UML - software modelling                   | OMG BPMN - business process modelling                 |
| Typical scope       | The logic inside one use case or component | An end-to-end business process                        |
| Participants        | Partitions, used less often                | Pools and lanes, used constantly                      |
| Events and messages | Limited                                    | Rich - message, timer, error, signal, boundary events |
| Audience            | Developers and analysts                    | Business stakeholders and developers                  |

#### EXAM / INTERVIEW TIP

If asked to distinguish them in an interview: they overlap heavily and both show flow with decisions. **BPMN is the business-facing standard built around participants and events; activity diagrams are the UML notation for detailed behaviour, strongest where concurrency matters.** Most organisations pick one and use it for everything.

### 3.14 The Pairs People Confuse

| Pair                       | The distinction                                                                                                                                                        |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BPMN vs swimlane flowchart | Same shape on the page, different rigour. BPMN symbols have fixed meanings and rules; a swimlane flowchart is a convention                                             |
| BPMN vs activity diagram   | Business process standard versus UML behaviour notation. BPMN is stronger on participants and events; activity diagrams are stronger on concurrency inside a component |
| DFD vs BPMN                | DFD shows data movement with no sequence or decisions; BPMN shows control flow with both. A DFD can never answer 'what happens next'                                   |
| Use case vs user story     | A use case specifies a complete interaction including alternative and exception flows; a user story is a placeholder for a conversation, sized to fit in a sprint      |
| ERD vs class diagram       | ERD models data and relationships for storage; a UML class diagram also models behaviour (methods) and is a software design artefact                                   |

| Pair                           | The distinction                                                                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| State diagram vs process model | A state diagram tracks what one object is; a process model tracks what people and systems do. Nouns versus verbs              |
| Context diagram vs level-1 DFD | The context diagram is one process bubble showing the boundary; level 1 opens that bubble into its major processes and stores |

**KEY POINT**

A useful closing line when you present this to your team: **the diagrams are not competing - they are different cross-sections of the same feature.** The use case says who and what, BPMN says how and in what order, the ERD says what data makes it possible, the state diagram says what is legal, the wireframe says what it looks like, and the RTM proves it all got built.

## PART 4

# SQL Essentials for Analysts

*Reading data yourself instead of queuing for someone who can*

## 4

## What this part covers

- Why a BA who can query is worth more than one who cannot
- The anatomy of a SELECT and the order the database actually runs it in
- Filtering, sorting, grouping and the WHERE versus HAVING distinction
- Joins, with the four types and when each is right
- The Northwind schema you will practise against
- Five business questions answered with working queries
- The mistakes that silently produce wrong numbers

## 4.1 Why This Matters

*A BA who can run a query answers 'how often does that actually happen?' in ten minutes. A BA who cannot raises a ticket and waits three days for an answer to the wrong question.*

- **Evidence beats opinion.** 'Customers reorder all the time' becomes '38 percent of orders in the last quarter repeated a previous basket exactly'.
- **Edge cases become visible.** Someone will tell you an order always has at least one line. One query tells you whether that is true.
- **Requirements get sharper.** Knowing the actual distribution of order sizes changes what you specify for pagination, performance and display.
- **You stop being a bottleneck.** Most BA data questions are a single SELECT with a join and a GROUP BY.

### KEY POINT

You do not need to write production SQL. You need to **read** a schema and **answer a business question** with a query you would be comfortable showing to a data engineer. That is roughly six keywords.

## 4.2 Anatomy of a SELECT



```

SELECT    c.Country, COUNT(*) AS OrderCount    -- 5. what to show
FROM      Orders o                            -- 1. from where
JOIN      Customers c ON c.CustomerID = o.CustomerID
WHERE     o.OrderDate >= '1997-01-01'         -- 2. filter rows
GROUP BY  c.Country                          -- 3. collapse into groups
HAVING    COUNT(*) > 10                      -- 4. filter groups
ORDER BY  OrderCount DESC                    -- 6. sort
LIMIT     5;                                -- 7. trim

```

The clauses are **written** in that order but the database **executes** them roughly in the numbered order shown. That single fact explains most beginner errors.

| Runs | Clause      | What it does                                  |
|------|-------------|-----------------------------------------------|
| 1    | FROM / JOIN | Assembles the working set of rows             |
| 2    | WHERE       | Filters individual rows - before any grouping |
| 3    | GROUP BY    | Collapses rows into groups                    |
| 4    | HAVING      | Filters the groups - after aggregation        |
| 5    | SELECT      | Chooses and computes the output columns       |
| 6    | ORDER BY    | Sorts the result                              |
| 7    | LIMIT / TOP | Returns only the first n rows                 |

#### WATCH OUT

Because SELECT runs after GROUP BY, you cannot filter on an aggregate in WHERE. **WHERE filters rows, HAVING filters groups.** 'Customers who spent over 10,000' is a HAVING; 'orders placed in 1997' is a WHERE. Getting this backwards is the single most common SQL interview failure.

## 4.3 Filtering, Sorting and NULLs

| Operator   | Meaning                                                               | Example                                         |
|------------|-----------------------------------------------------------------------|-------------------------------------------------|
| = and !=   | Equal, not equal                                                      | Country = 'Germany'                             |
| BETWEEN    | Inclusive range                                                       | OrderDate BETWEEN '1997-01-01' AND '1997-12-31' |
| IN         | Matches any value in a list                                           | Country IN ('UK','France','Spain')              |
| LIKE       | Pattern match; percent is any string, underscore any single character | CompanyName LIKE 'A%'                           |
| IS NULL    | Has no value at all                                                   | ShippedDate IS NULL                             |
| AND OR NOT | Combine conditions - use brackets when mixing                         | a AND (b OR c)                                  |

**WATCH OUT**

**NULL is not zero and not an empty string.** It means unknown. *ShippedDate = NULL* matches nothing, ever - you must write *ShippedDate IS NULL*. NULLs are also skipped by AVG and COUNT(column), which quietly changes your averages.

- **ORDER BY column DESC** sorts descending; default is ascending.
- Row limiting differs by database: **LIMIT 5** in PostgreSQL, MySQL and SQLite; **SELECT TOP 5** in SQL Server; **FETCH FIRST 5 ROWS ONLY** in Oracle and standard SQL.
- **COALESCE(a, b)** returns the first non-null value - useful for tidying reports.
- **DISTINCT** removes duplicate rows; reach for it only after checking why duplicates appeared, because usually a join is the real cause.

## 4.4 Aggregates and Grouping

| Function            | Returns                                  | Note                                           |
|---------------------|------------------------------------------|------------------------------------------------|
| COUNT(*)            | Number of rows                           | Counts every row including those with NULLs    |
| COUNT(col)          | Number of non-null values in that column | Different from COUNT(*)                        |
| COUNT(DISTINCT col) | Number of different values               | How you count unique customers                 |
| SUM(col)            | Total                                    | Ignores NULLs                                  |
| AVG(col)            | Mean                                     | Ignores NULLs - which may not be what you want |
| MIN / MAX(col)      | Smallest / largest                       | Works on dates and text too                    |

**KEY POINT**

The rule for GROUP BY: **every non-aggregated column in your SELECT must appear in the GROUP BY.** If you select customer name and SUM of value, you must group by customer name. Databases that let you break this rule are doing you no favours.

## 4.5 Joins



**INNER JOIN**  
rows matching in both



**LEFT JOIN**  
all of A, matches from B



**RIGHT JOIN**  
all of B, matches from A



**FULL OUTER JOIN**  
everything from both

*Which rows survive each join type*

| Join            | Keeps                                        | Use when                                                    |
|-----------------|----------------------------------------------|-------------------------------------------------------------|
| INNER JOIN      | Only rows with a match on both sides         | You want customers who have orders                          |
| LEFT JOIN       | All rows from the left table, matched or not | You want all customers, including those with no orders      |
| RIGHT JOIN      | All rows from the right table                | Rare in practice - most people swap the tables and use LEFT |
| FULL OUTER JOIN | Everything from both sides                   | Reconciling two systems                                     |
| SELF JOIN       | A table joined to itself                     | Employee to manager, or comparing rows in one table         |

**EXAM / INTERVIEW TIP**

The **LEFT JOIN with IS NULL** pattern is the one a BA uses most: join the left table to the right, then keep only rows where the right side came back empty. That answers every 'who has not done X' question - customers who never ordered, orders never shipped, products never sold.

## 4.6 The Northwind Schema

Northwind is a fictional food wholesaler - conveniently close to a delivery business. These are the tables you will need.

| Table         | Key columns                                                                                          | Relates to                           |
|---------------|------------------------------------------------------------------------------------------------------|--------------------------------------|
| Customers     | CustomerID (PK), CompanyName, ContactName, City, Country                                             | Orders                               |
| Orders        | OrderID (PK), CustomerID (FK), EmployeeID (FK), OrderDate, ShippedDate, ShipCountry                  | Customers, Employees, Order Details  |
| Order Details | OrderID (FK), ProductID (FK), UnitPrice, Quantity, Discount                                          | Orders, Products                     |
| Products      | ProductID (PK), ProductName, CategoryID (FK), SupplierID (FK), UnitPrice, UnitsInStock, Discontinued | Categories, Suppliers, Order Details |
| Categories    | CategoryID (PK), CategoryName                                                                        | Products                             |
| Employees     | EmployeeID (PK), LastName, FirstName, ReportsTo                                                      | Orders, Employees (self)             |
| Suppliers     | SupplierID (PK), CompanyName, Country                                                                | Products                             |

**WATCH OUT**

Two Northwind traps. First, the order-details table is named **Order Details** with a space in the Microsoft versions, so it must be bracketed or quoted: [Order Details] in SQL Server, "Order Details" in PostgreSQL. Some ports rename it OrderDetails. Second, there is **no total column** anywhere - line value is always  $UnitPrice * Quantity * (1 - Discount)$ , and you must compute it every time.

## 4.7 Five Worked Queries

Each one starts from a business question, because that is how they will arrive in real life.

### Query 1 · Top 5 customers by total order value

**Business question:** who are our most valuable customers, so we know whose behaviour to protect when we change the ordering journey?

```
SELECT      c.CustomerID,
            c.CompanyName,
            c.Country,
            COUNT(DISTINCT o.OrderID)                AS OrderCount,
            ROUND(SUM(od.UnitPrice * od.Quantity *
                    (1 - od.Discount)), 2)           AS TotalValue
FROM        Customers c
JOIN        Orders o ON o.CustomerID = c.CustomerID
JOIN        "Order Details" od ON od.OrderID = o.OrderID
GROUP BY    c.CustomerID, c.CompanyName, c.Country
ORDER BY    TotalValue DESC
LIMIT       5;
```

- Three tables because value lives on the line items, not on the order.
- **COUNT(DISTINCT o.OrderID)** not COUNT(\*) - joining to line items multiplies each order row once per line, so a plain count would report line items and call them orders.
- All three non-aggregated columns appear in GROUP BY.
- Swap LIMIT 5 for SELECT TOP 5 on SQL Server.

### Query 2 · Monthly revenue trend

**Business question:** is revenue growing, and is there seasonality we should plan capacity around?

```
SELECT      STRFTIME('%Y-%m', o.OrderDate)           AS OrderMonth,
            COUNT(DISTINCT o.OrderID)                AS Orders,
            ROUND(SUM(od.UnitPrice * od.Quantity *
                    (1 - od.Discount)), 2)           AS Revenue
FROM        Orders o
JOIN        "Order Details" od ON od.OrderID = o.OrderID
WHERE       o.OrderDate >= '1997-01-01'
            AND o.OrderDate < '1998-01-01'
GROUP BY    OrderMonth
ORDER BY    OrderMonth;
```

- Date formatting is the most database-specific thing in SQL. **STRFTIME** is SQLite; use **TO\_CHAR(o.OrderDate, 'YYYY-MM')** in PostgreSQL and Oracle, **FORMAT(o.OrderDate, 'yyyy-MM')** or **DATEPART** in SQL Server, **DATE\_FORMAT(o.OrderDate, '%Y-%m')** in MySQL.
- Note the date range is written as **greater-or-equal to the start and strictly less than the next year**. That pattern is safe whether or not the column carries a time component; **BETWEEN** with a date can silently drop the last day.

### Query 3 · Top 10 products by revenue, with category

**Business question:** which products drive revenue, so we know what must never be unavailable on a reorder?

```
SELECT    p.ProductName,
          cat.CategoryName,
          SUM(od.Quantity)                                AS UnitsSold,
          ROUND(SUM(od.UnitPrice * od.Quantity *
                    (1 - od.Discount)), 2)                AS Revenue,
          p.Discontinued
FROM      "Order Details" od
JOIN      Products   p   ON p.ProductID = od.ProductID
JOIN      Categories  cat ON cat.CategoryID = p.CategoryID
GROUP BY  p.ProductName, cat.CategoryName, p.Discontinued
ORDER BY  Revenue DESC
LIMIT     10;
```

#### EXAMPLE

Including **Discontinued** is the analyst's touch. A high-revenue product that is now discontinued is exactly the item that will break a reorder - it sold well, so it is in lots of order histories, and it can no longer be supplied. That one column turns a sales report into a requirements insight.

### Query 4 · Customers who reorder the same product

**Business question:** how much genuine repeat behaviour exists? This is the evidence that justifies building a reorder feature at all.

```
SELECT    c.CompanyName,
          p.ProductName,
          COUNT(DISTINCT o.OrderID)                    AS TimesOrdered,
          MIN(o.OrderDate)                             AS FirstOrdered,
          MAX(o.OrderDate)                             AS LastOrdered
FROM      Customers c
JOIN      Orders     o  ON o.CustomerID = c.CustomerID
JOIN      "Order Details" od ON od.OrderID = o.OrderID
JOIN      Products   p  ON p.ProductID = od.ProductID
GROUP BY  c.CompanyName, p.ProductName
HAVING    COUNT(DISTINCT o.OrderID) >= 3
ORDER BY  TimesOrdered DESC, c.CompanyName;
```

- **HAVING**, not **WHERE**, because the filter is on a count that only exists after grouping.
- **MIN** and **MAX** of the order date give you the span of the habit - a customer who bought the same product five times across two years behaves differently from one who bought it five times in a month.
- This is the query that turns 'people reorder things' into a number you can put in a business case.

### Query 5 · Customers with no orders at all

**Business question:** who registered but never ordered? A reorder feature does nothing for them, so we should know how big that group is before claiming the feature lifts overall activity.

```
SELECT    c.CustomerID,  
          c.CompanyName,  
          c.Country  
FROM      Customers c  
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID  
WHERE     o.OrderID IS NULL  
ORDER BY  c.Country, c.CompanyName;
```

- The **LEFT JOIN plus IS NULL** pattern from 4.5. The left join keeps every customer; the WHERE then keeps only those where no matching order was found.
- The equivalent using NOT EXISTS is often clearer and usually performs at least as well:

```
SELECT    c.CustomerID, c.CompanyName, c.Country  
FROM      Customers c  
WHERE     NOT EXISTS (SELECT 1 FROM Orders o  
                     WHERE o.CustomerID = c.CustomerID);
```

#### WATCH OUT

Do not use **NOT IN** with a subquery on a column that can contain NULL. If a single NULL appears in the subquery result, NOT IN returns no rows at all, silently. NOT EXISTS does not have this problem.

## 4.8 Mistakes That Produce Wrong Numbers Quietly

| Mistake                                 | Symptom                                  | Fix                                                           |
|-----------------------------------------|------------------------------------------|---------------------------------------------------------------|
| COUNT(*) after joining to a child table | Order counts far too high                | COUNT(DISTINCT parent_id)                                     |
| Filtering an outer join in WHERE        | The LEFT JOIN behaves like an INNER JOIN | Move the condition into the ON clause                         |
| Comparing to NULL with equals           | Zero rows returned and no error          | Use IS NULL / IS NOT NULL                                     |
| BETWEEN on a datetime column            | The last day is missing                  | Use greater-or-equal start AND strictly-less-than next period |
| Averaging a column containing NULLs     | Average higher than expected             | Decide whether NULL means zero; use COALESCE if it does       |
| Forgetting Discount in Northwind        | Revenue overstated                       | Always UnitPrice * Quantity * (1 - Discount)                  |
| Rounding before summing                 | Totals drift by cents                    | SUM first, then ROUND                                         |

### KEY POINT

Sanity-check every result before you quote it. Does the row count look plausible? Does the total revenue roughly match a figure you already trust? Run the query without the GROUP BY and eyeball twenty rows. A confidently quoted wrong number costs more credibility than saying 'I don't know yet'.

## 4.9 Going Further

- **Subqueries** - a query inside a query, in the WHERE clause or in the FROM clause as a derived table.
- **Common table expressions** - WITH name AS (...) SELECT ... FROM name. Far more readable than nested subqueries, and the single biggest readability upgrade you can make.
- **CASE WHEN** - conditional logic inside a SELECT, for bucketing values into bands.
- **Window functions** - ROW\_NUMBER, RANK, SUM OVER. These answer 'top n per group' and 'running total' questions that are painful otherwise.
- **Practice environments** - SQLite with the Northwind file needs no server; DB Fiddle, SQLZoo and Mode's SQL tutorial run entirely in a browser.

### EXAM / INTERVIEW TIP

Practise by writing the **business question first**, in one sentence, then the query. That habit is what separates an analyst from someone who knows SQL syntax - and it is exactly how the question will be asked in an interview.

# Self-Check Questions

*Cover the answer, say yours aloud, then compare.*

## The artefact chain

### What is a business capability, and how is it different from a process?

A capability is what the business is able to do, stated as a noun phrase and independent of how, who or what technology. A process is how that capability is exercised, step by step. Capabilities are stable; processes change around them.

### Name the three levels of business process and who reads each.

Level 1, the process landscape or value chain, for executives. Level 2, the process map or sub-process, for process owners. Level 3, the detailed process flow with roles and systems, for the BA and delivery team.

### How do you know you have mixed levels in one diagram?

Two boxes on the same diagram would take wildly different amounts of time - 'Manage orders' sitting next to 'Click Confirm'.

### What makes a persona useful rather than decorative?

It is based on research rather than opinion, it states goals, frustrations and constraints, and it includes anti-goals. It should change a design decision; if it cannot, it is decoration.

## BPMN

### What is the difference between sequence flow and message flow?

Sequence flow is a solid arrow connecting activities within a single pool. Message flow is a dashed arrow with an open head connecting two pools. Sequence flow must never cross a pool boundary.

### When do you use an exclusive gateway versus a parallel one?

Exclusive when exactly one path is taken because the conditions are mutually exclusive. Parallel when all paths are taken at once because the work is independent.

### Why should a gateway never contain a verb?

Gateways route, they do not perform work. A verb means you need a task before the gateway.

### What is the most common omission in a first-draft process model?

The exception paths. A happy-path-only model has not been analysed, and the requirements you were looking for are mostly in the exceptions.

### What is the difference between dynamic and point glue in Visio?

Dynamic glue attaches to the whole shape and re-routes when shapes move; point glue attaches to a specific handle and does not. Use dynamic glue for anything you might rearrange.

## Diagrams

### Include versus extend on a use case diagram?

Include is always - the base use case always performs the included one. Extend is sometimes - the extending use case adds behaviour only under a condition.



**What can a DFD never tell you?**

Sequence or decisions. It shows data movement between processes, stores and external entities, with no control flow at all.

**What is the value of the transitions you do not draw on a state diagram?**

They are requirements. No arrow from Delivered back to Preparing means the system must prevent that transition - a requirement nobody would have written in prose.

**How does an ERD support the reorder feature specifically?**

By storing unit price on the order line as well as referencing the product, so you can compare what the customer paid last time with what it costs now and tell them the price changed.

**Why is a detailed Gantt of user stories an agile anti-pattern?**

It re-fixes the scope that an adaptive approach exists to keep flexible. Use a roadmap of outcomes by quarter instead.

## SQL

**WHERE versus HAVING?**

WHERE filters individual rows before grouping. HAVING filters groups after aggregation. 'Orders in 1997' is WHERE; 'customers with more than ten orders' is HAVING.

**Why is COUNT(\*) wrong when you have joined orders to order lines?**

The join produces one row per line item, so COUNT(\*) counts line items. Use COUNT(DISTINCT OrderID) to count orders.

**How do you find customers who have never ordered?**

LEFT JOIN customers to orders and keep rows where the order key IS NULL, or use NOT EXISTS with a correlated subquery.

**Why does ShippedDate = NULL return nothing?**

NULL means unknown, so it is never equal to anything, including itself. Use IS NULL.

**What must every non-aggregated column in a SELECT also appear in?**

The GROUP BY clause.

**How is line value calculated in Northwind?**

UnitPrice multiplied by Quantity multiplied by one minus Discount. There is no stored total column.

**EXAM / INTERVIEW TIP**

Before your mentor session, be ready to **walk through your BPMN model out loud**, lane by lane, and to explain one query line by line. Being able to narrate an artefact is a different skill from producing it, and it is the one that gets tested in real meetings.